Deep Neural Networks for Survival Analysis with survdnn

Architecture, loss functions, and prediction in a native R torch implementation.

Imad EL BADISY

2026-08-24

Table of contents

Right-censored survival data $\{(t_i, \delta_i, x_i)\}_{i=1}^n$ pair an observed time $t_i = \min(T_i, C_i)$ and an event indicator $\delta_i = \mathbb{1}(T_i \leq C_i)$ with a covariate vector $x_i \in \mathbb{R}^p$. The Cox proportional hazards model expresses the hazard as $\lambda(t \mid x) = \lambda_0(t) \exp(\beta^\top x)$, restricting covariate effects to a linear,

time-constant log-hazard ratio. `survdnn` replaces the linear predictor $\beta^\top x$ with a multilayer perceptron $f_\theta(x)$, keeping the estimation machinery around it (partial likelihood, Breslow baseline, time-dependent risk) intact, and extends the same replacement to non-proportional-hazards and accelerated failure time formulations.[1] The package is implemented natively in R on top of `torch`, avoiding a Python backend.[2]

Architecture

This section is deliberately exhaustive: every transform between raw covariates and the scalar output $f_\theta(x)$ is written out, including the pieces (initialization, batch normalization's train/eval distinction, dropout's exact scaling) that are usually left implicit because a deep learning framework handles them silently. The intent is that this note, not the `torch`/`survdnn` source, should be a sufficient reference for extending or re-deriving any part of the architecture.

From formula to design matrix

Fitting starts from an R survival formula, `Surv(time, status) ~ x1 + x2 + ...`. `stats::model.frame()` and `stats::model.matrix()` turn this into a numeric design matrix exactly as `lm()`/`coxph()` would: factor covariates are expanded into indicator (dummy) columns via the formula's contrasts, and the intercept column that `model.matrix()` adds by default is dropped (`[, -1]`), since a survival network has no free intercept term of its own, the loss functions below absorb any constant shift into the network's output or into an explicit location term (AFT's c). Write p for the resulting number of numeric predictor columns after expansion, so each row is $x_i \in \mathbb{R}^p$.

Standardization

Before any layer sees the data, each column of the design matrix is z-scored using **training-data moments only**:

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}, \qquad \bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}, \qquad s_j = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)^2},$$

the usual sample mean and unbiased sample standard deviation, one pair $(\bar{x}_j, s_j)$ per column, computed once from the training set (R's `scale()`). These p pairs are stored on the fitted object (`x_center`, `x_scale`) and reused, never recomputed, on any later call to `predict()`: this is what makes prediction on new data well defined without leaking new-data information into the scaling, exactly analogous to storing a training-only imputation model rather than

[1] `survdnn` package documentation, CRAN: https://CRAN.R-project.org/package=survdnn.
[2] EL BADISY, I. (2026). *SurvDNN: Survival Deep Learning Models for Tabular Data*. The R Journal.

re-fitting one on test data. Standardization exists purely to help gradient-based optimization (unscaled covariates on very different numeric ranges produce ill-conditioned loss surfaces); it changes nothing about what the network can represent, only how easily it is fit.

Layer transform

For hidden layer sizes $h_1, \dots, h_L$ (the `hidden` argument, e.g. `c(32, 16)` means $L = 2$, $h_1 = 32$, $h_2 = 16$), define $z^{(0)} = \tilde{x} \in \mathbb{R}^p$ and, for $l = 1, \dots, L$:

$$u^{(l)} = W^{(l)} z^{(l-1)} + b^{(l)} \in \mathbb{R}^{h_l}, \qquad W^{(l)} \in \mathbb{R}^{h_l \times h_{l-1}},\ b^{(l)} \in \mathbb{R}^{h_l}$$

$$v^{(l)} = \mathrm{BN}\big(u^{(l)}\big) \quad (\text{if } \texttt{batch\_norm = TRUE}, \text{ else } v^{(l)} = u^{(l)})$$

$$w^{(l)} = \phi\big(v^{(l)}\big), \qquad z^{(l)} = \mathrm{Dropout}\big(w^{(l)}\big) \quad (\text{if } \texttt{dropout > 0}, \text{ else } z^{(l)} = w^{(l)}).$$

The output layer is a further, separate linear map with no activation, batch norm, or dropout after it:

$$f_\theta(x) = W^{(L+1)} z^{(L)} + b^{(L+1)} \in \mathbb{R}, \qquad W^{(L+1)} \in \mathbb{R}^{1 \times h_L},\ b^{(L+1)} \in \mathbb{R}.$$

$\theta = (W^{(1)}, b^{(1)}, \dots, W^{(L+1)}, b^{(L+1)})$ collects every trainable parameter; for the Cox-Time loss, $z^{(0)}$ is instead $(\tilde{t}, \tilde{x}) \in \mathbb{R}^{p+1}$ with time standardized the same way as the covariates (Section "Cox-Time" below), so $p_{\text{input}} = p + 1$ rather than p.

Weight initialization

Each $W^{(l)}$ and $b^{(l)}$ is initialized independently by `torch`'s default for a linear layer, Kaiming-uniform with the fan-in of that layer:

$$W^{(l)}_{ab},\ b^{(l)}_a \overset{\text{iid}}{\sim} \mathrm{Uniform}\left(-\frac{1}{\sqrt{h_{l-1}}},\ \frac{1}{\sqrt{h_{l-1}}}\right),$$

so a wider incoming layer gets a narrower initial weight range, keeping the initial output variance roughly stable across layers of different widths. This is `torch`'s library default, not a `survdnn`-specific choice; `survdnn` does not override it. Because initialization is random, `.seed` is what makes a fit reproducible: it seeds both R's and `torch`'s random number generators, which otherwise draw independently.

Activation functions

ϕ is applied elementwise and is one of seven options, each `torch`'s standard definition at its default hyperparameters:

$$
\begin{aligned}
\texttt{relu}(v) &= \max(0, v) \\
\texttt{leaky\_relu}(v) &= \begin{cases} v & v \geq 0 \\ 0.01\, v & v < 0 \end{cases} \\
\texttt{tanh}(v) &= \frac{e^v - e^{-v}}{e^v + e^{-v}} \\
\texttt{sigmoid}(v) &= \frac{1}{1 + e^{-v}} \\
\texttt{gelu}(v) &= v\,\Phi(v), \qquad \Phi = \text{standard normal CDF (exact, not the tanh-approximation variant)} \\
\texttt{elu}(v) &= \begin{cases} v & v \geq 0 \\ e^v - 1 & v < 0 \end{cases} \\
\texttt{softplus}(v) &= \log(1 + e^v)
\end{aligned}
$$

The same ϕ is used at every hidden layer; `survdnn` does not support mixing activations across layers within one model.

Batch normalization

When `batch_norm = TRUE`, each pre-activation coordinate $u_a^{(l)}$ $(a = 1, \ldots, h_l)$ is normalized **per coordinate, across the batch dimension**, then rescaled by a learnable affine transform:

$$
\mathrm{BN}(u_a^{(l)}) = \gamma_a^{(l)} \cdot \frac{u_a^{(l)} - \mu_a}{\sqrt{\sigma_a^2 + \varepsilon}} + \beta_a^{(l)}, \qquad \varepsilon = 10^{-5},
$$

where $\gamma_a^{(l)}, \beta_a^{(l)}$ are trainable parameters, initialized to 1 and 0 respectively, so batch norm starts as an identity map on the standardized scale and only deviates from it as training updates γ, β. Since `survdnn` always trains full-batch (Section "Training" below), μ_a, σ_a^2 during training are the mean and (biased) variance of $u_a^{(l)}$ **over the entire training set**, recomputed every epoch as the parameters feeding into $u^{(l)}$ change, not a stochastic mini-batch estimate as batch norm is usually described. In parallel, an exponential running estimate is accumulated every epoch,

$$
\bar{\mu}_a \leftarrow (1 - m)\,\bar{\mu}_a + m\,\mu_a, \qquad \bar{\sigma}_a^2 \leftarrow (1 - m)\,\bar{\sigma}_a^2 + m\,\sigma_a^2, \qquad m = 0.1,
$$

and it is $\bar{\mu}_a, \bar{\sigma}_a^2$, not the batch statistics, that `predict()` uses (the layer is switched to evaluation mode, `net$eval()`, before prediction). For full-batch training this running estimate converges toward the final epochs' batch statistics as training proceeds, but is not identical to the very last epoch's batch statistics, a small, generally negligible source of train/predict discrepancy worth knowing about if a fitted model's predictions on its own training data are compared bit-for-bit against the final training loss.

Dropout

When `dropout > 0` (default 0.3), each coordinate of $w^{(l)}$ is independently zeroed with probability $p_{\text{drop}} =$ `dropout` during training, and the surviving coordinates are rescaled up so the layer's expected output is unchanged (inverted dropout):

$$z_a^{(l)} = \frac{m_a}{1 - p_{\text{drop}}} w_a^{(l)}, \qquad m_a \overset{\text{iid}}{\sim} \text{Bernoulli}(1 - p_{\text{drop}}),$$

drawn fresh for every forward pass during training. At evaluation time (`net$eval()`, used throughout `predict()`) dropout is switched off entirely, $z^{(l)} = w^{(l)}$, with no compensating scale factor needed since the training-time rescaling already made the layer's expected output consistent between the two modes.

Putting it together

$f_\theta(x) = g_\theta(z^{(L)}(\tilde{x}))$ where g_θ is the final linear map and $z^{(L)}(\tilde{x})$ is the composition of L blocks above, each itself a composition of an affine map, an optional batch-norm affine renormalization, a fixed elementwise nonlinearity, and an optional stochastic masking. The scalar output $f_\theta(x)$ has no fixed meaning on its own: it is a log-risk score under the two Cox losses, a location parameter on the log-time scale under AFT, and a time-dependent log-risk score under Cox-Time. Everything downstream, including how a fitted network becomes a survival curve, depends on which of these it is, and is worked out loss by loss below.

Loss functions

Cox partial likelihood

Under proportional hazards, $\lambda(t \mid x) = \lambda_0(t)e^\eta$ with $\eta = f_\theta(x)$. Cox's partial likelihood avoids ever estimating $\lambda_0(t)$ by conditioning on which subject, among those still at risk, is the one who fails at each observed event time. For a risk set $R(t_i) = \{j : t_j \geq t_i\}$, the probability that it is subject i who fails, given that exactly one failure occurs at t_i, is

$$\Pr(i \text{ fails} \mid \text{one failure in } R(t_i)) = \frac{\lambda_0(t_i)e^{\eta_i}}{\sum_{j \in R(t_i)} \lambda_0(t_i)e^{\eta_j}} = \frac{e^{\eta_i}}{\sum_{j \in R(t_i)} e^{\eta_j}},$$

with $\lambda_0(t_i)$ canceling top and bottom, which is exactly why no baseline hazard model is needed to train on this objective. `survdnn` trains f_θ against the negative partial log-likelihood, averaged over events,

$$\ell_{\text{cox}}(\theta) = -\frac{1}{n_e} \sum_{i:\, \delta_i = 1} \left(f_\theta(x_i) - \log \sum_{j \in R(t_i)} \exp f_\theta(x_j) \right),$$

where n_e is the number of events. The inner sum is computed with `torch_logcumsumexp()` on time-sorted linear predictors, so risk-set membership falls out of the sort order and the log-sum-exp stays numerically stable rather than overflowing. An L2 variant, `"cox_l2"`, adds a penalty directly on the predicted risk scores rather than on the network weights,

$$\ell_{\text{cox-l2}}(\theta) = \ell_{\text{cox}}(\theta) + \lambda \cdot \frac{1}{n} \sum_{i=1}^{n} f_\theta(x_i)^2, \qquad \lambda = 10^{-3}.$$

Penalizing the output constrains what the network is allowed to predict regardless of how many parameters produced it, which is a more direct fix for the instability that a few extreme risk scores cause in every risk-set sum above, compared to ordinary weight decay acting on the parameters themselves.

Accelerated failure time

Under the AFT loss, the network output $\mu_\theta(x)$ is the location parameter of a log-normal model for the event time,

$$\log T = c + \mu_\theta(x) + \sigma Z, \qquad Z \sim \mathcal{N}(0, 1),$$

where c is a location offset fixed, before training, at the mean log event time among observed events in the training data (stored as `aft_loc`), and $\sigma > 0$ is a single global scale parameter optimized jointly with θ through the reparameterization $\sigma = \exp(\log \sigma)$, which keeps it positive without a constrained optimizer. Centering on c keeps the network's job restricted to a zero-centered deviation instead of also having to represent the population-average log-time through its bias term. With $z_i = (\log t_i - c - \mu_\theta(x_i))/\sigma$, the censored negative log-likelihood combines an exact density term for events with a survival term for censored observations,

$$\ell_{\text{aft}}(\theta, \sigma) = \frac{1}{n} \sum_{i=1}^{n} \Big[\delta_i \big(\log t_i + \log \sigma + \tfrac{1}{2} z_i^2 \big) - (1 - \delta_i) \log S_Z(z_i) \Big], \qquad S_Z(z) = 1 - \Phi(z),$$

the standard parametric censored-data likelihood, with the location term replaced by a network.

Cox-Time

The Cox-Time loss extends the Cox partial likelihood to a time-varying log-risk function $g_\theta(t, x)$, taking time as an explicit network input rather than treating risk as fixed per subject.[3] The contribution of an event at t_i becomes

$$\ell_{\text{coxtime}}(\theta) = -\frac{1}{n_e} \sum_{i:\, \delta_i = 1} \left(g_\theta(t_i, x_i) - \log \sum_{j \in R(t_i)} \exp\, g_\theta(t_i, x_j) \right),$$

which differs from the Cox loss only in that the denominator re-evaluates the network at t_i for every subject at risk, instead of reusing one risk score per subject computed once. This is what buys freedom from proportional hazards, and also what makes the loss expensive: building the denominator at every event time requires one forward pass per (event time, at-risk subject) pair, so training cost scales with the number of events rather than staying fixed as it does for plain Cox. Risk-set membership is always computed on raw observed time, while the time fed into the network is centered and scaled for optimization stability, exactly mirroring how covariates are standardized before entering the network.

Prediction

`predict.survdnn()` returns one of three things depending on `type`. `type = "lp"` returns the raw network output as interpreted by the training loss. `type = "survival"` returns predicted survival probabilities at a set of times. `type = "risk"` returns $1 - S(t)$ at a single time point. The mechanics of getting from a trained network to an actual curve differ completely across the four losses, because only AFT trains a fully specified parametric model; the others train a relative or time-varying risk score that still needs a baseline hazard attached.

For Cox and Cox-L2, survival curves are not read directly off the network. Predictions require refitting a one-covariate `coxph()` on the training linear predictors and extracting its Breslow cumulative baseline hazard $\hat{H}_0(t)$ via `survival::basehaz()`, interpolated at the

[3] Kvamme, H., Borgan, Ø., and Scheel, I. (2019). Time-to-event prediction with neural networks and Cox regression. *Journal of Machine Learning Research*, 20(129), 1-30.

requested times. This reuses `coxph()`'s existing risk-set summation and tie-handling rather than reimplementing the Breslow estimator directly against the network's gradients:

$$\hat{S}(t \mid x) = \exp\big(-\hat{H}_0(t)\exp(f_\theta(x))\big).$$

For AFT, survival probabilities follow directly from the fitted log-normal model, with no post-hoc refitting step, using the stored c and the learned $\hat{\sigma}$ from training:

$$\hat{S}(t \mid x) = 1 - \Phi\left(\frac{\log t - c - \mu_\theta(x)}{\hat{\sigma}}\right).$$

For Cox-Time, there is no closed-form Breslow shortcut, since the baseline hazard is only meaningful jointly with a time-varying g_θ. Instead, baseline hazard increments are estimated at each observed training event time t_k from the training risk sets,

$$d\hat{H}_0(t_k) = \frac{dN(t_k)}{\sum_{j \in R(t_k)} \exp\, g_\theta(t_k, x_j)},$$

a discrete Nelson-Aalen-style ratio of events at t_k to risk-weighted exposure at t_k, and cumulative hazard at a query time is obtained by summing increments at all event times up to that point, each evaluated through the new subject's own network output at that grid point:

$$\hat{H}(t \mid x) = \sum_{t_k \le t} \exp\, g_\theta(t_k, x)\, d\hat{H}_0(t_k), \qquad \hat{S}(t \mid x) = \exp\big(-\hat{H}(t \mid x)\big).$$

This requires one forward pass per event time per subject on both the training side, to build $d\hat{H}_0$, and the prediction side, which is the direct computational cost of letting the risk score vary with time.

The Cox and Cox-L2 losses share exactly this same prediction path: nothing downstream of the frozen network can tell them apart, since the output-level penalty in `"cox_l2"` only changes what happens during training, not the Breslow step above. AFT and Cox-Time, in contrast, each need their own prediction formula because their trained scalar means something structurally different.

Training

Loop structure

For each of the `epochs` iterations: a forward pass computes $f_\theta(x_i)$ for every training subject i and the scalar loss $\ell(\theta)$ (Cox, Cox-L2, AFT, or Cox-Time, all defined below); `loss$backward()` runs reverse-mode automatic differentiation to populate $\nabla_\theta \ell$ on every parameter tensor; the optimizer's `step()` then updates θ using that gradient; and `zero_grad()` clears the accumulated gradient before the next epoch, since `torch`, like PyTorch, accumulates gradients across calls by default rather than overwriting them. Training is full-batch, not mini-batch: every epoch's forward and backward pass uses the entire training set at once, so "one epoch" and "one optimizer step" are the same thing here, unlike stochastic mini-batch training where many optimizer steps occur per epoch. This is also why the batch-normalization statistics in the previous section are exact full-dataset statistics at every step rather than noisy mini-batch estimates.

Optimizer update rules

Write g_k for $\nabla_\theta \ell(\theta_k)$ at step k (one step = one epoch here), applied elementwise to every parameter tensor independently. All five optimizers share the same loop structure but differ in how they turn g_k into a parameter update $\theta_{k+1} = \theta_k - \eta \cdot$ (something built from g_k and its history), where η is the learning rate (`lr`, default 10^{-4}).

SGD (`"sgd"`) is the raw gradient step, $\theta_{k+1} = \theta_k - \eta\, g_k$ (with an optional momentum term if supplied via `optim_args`).

Adagrad (`"adagrad"`) accumulates the sum of squared gradients per parameter and scales the step down as that accumulator grows, $s_k = s_{k-1} + g_k^2$, $\theta_{k+1} = \theta_k - \dfrac{\eta}{\sqrt{s_k} + \varepsilon}\, g_k$ ($\varepsilon = 10^{-10}$, `torch`'s default), so parameters that have received large gradients so far get progressively smaller steps.

RMSprop (`"rmsprop"`) replaces Adagrad's ever-growing sum with an exponential moving average of squared gradients, $s_k = \alpha\, s_{k-1} + (1-\alpha) g_k^2$ (α the smoothing constant, `torch`'s default 0.99, its `optim_rmsprop()` argument name), $\theta_{k+1} = \theta_k - \dfrac{\eta}{\sqrt{s_k} + \varepsilon}\, g_k$ ($\varepsilon = 10^{-8}$), so the effective step size adapts to recent gradient magnitude rather than the full training history.

Adam (`"adam"`, the default) maintains exponential moving averages of both the gradient itself (first moment, m_k) and its square (second moment, v_k), each bias-corrected for their zero initialization:

$$m_k = \beta_1 m_{k-1} + (1-\beta_1) g_k, \qquad v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k^2, \qquad (\beta_1, \beta_2) = (0.9, 0.999) \text{ (torch defaults)},$$

$$\hat{m}_k = \frac{m_k}{1-\beta_1^k}, \qquad \hat{v}_k = \frac{v_k}{1-\beta_2^k}, \qquad \theta_{k+1} = \theta_k - \eta\,\frac{\hat{m}_k}{\sqrt{\hat{v}_k}+\varepsilon}, \qquad \varepsilon = 10^{-8}.$$

m_k smooths the gradient direction (momentum), v_k adapts the step size down for parameters with consistently large gradients, and the bias correction $1-\beta_i^k$ compensates for $m_0 = v_0 = 0$ making the raw averages biased toward zero at small k. If `weight_decay` is set (via `optim_args`, or the `1e-4` `survdnn` default whenever `optimizer %in% c("adam","adamw")` and the user did not override it), plain `"adam"` adds it as an L_2 penalty folded directly into g_k before the moment updates, $g_k \leftarrow g_k + \lambda\theta_k$.

AdamW (`"adamw"`) is otherwise identical to Adam but decouples weight decay from the gradient-based moment estimates, applying it as a separate multiplicative shrinkage at the update step, $\theta_{k+1} = \theta_k - \eta\left(\frac{\hat{m}_k}{\sqrt{\hat{v}_k}+\varepsilon} + \lambda\theta_k\right)$, which avoids weight decay being distorted by Adam's per-parameter adaptive scaling, the standard argument for preferring AdamW over Adam whenever nonzero weight decay is used.[4]

Callbacks, missing data, and reproducibility

Callbacks receive the epoch index and current loss after each step and return `TRUE`/`FALSE`; a `TRUE` halts training immediately, as with `callback_early_stopping()`, which is how early stopping is implemented, there is no separate built-in patience/tolerance mechanism outside this callback interface. Missing values in the model variables are either dropped, `na_action = "omit"` (rows with any `NA` in the formula's variables are excluded before any tensor is built, with a message reporting how many), or treated as a hard error, `na_action = "fail"`. Reproducibility across R and `torch`'s separate random number generators, which otherwise advance independently and are each responsible for a different part of the pipeline (R's for the train/validation splits used elsewhere in the package, `torch`'s for weight initialization and dropout masks), is handled by a single `.seed` argument that seeds both.

Evaluation

Three metrics are implemented directly against predicted survival matrices rather than delegated to an external package: the concordance index, `cindex_survmat()`, evaluated at a fixed horizon by comparing $1-S(t^\star \mid x)$ across all admissible pairs; the IPCW-weighted Brier score, `brier()`, at a single time point, using a Kaplan-Meier estimate of the censoring distribution as weights; and the integrated Brier score, `ibs_survmat()`, a trapezoidal integral of the Brier score over a grid of times, normalized by the width of the grid. `evaluate_survdnn()` wraps these for a fitted model, and `cv_survdnn()` repeats fit-and-evaluate over stratified folds, stratifying on

[4] Loshchilov, I., and Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR).*

the event indicator so that fold sizes stay balanced across censoring rates rather than across time-to-event values.

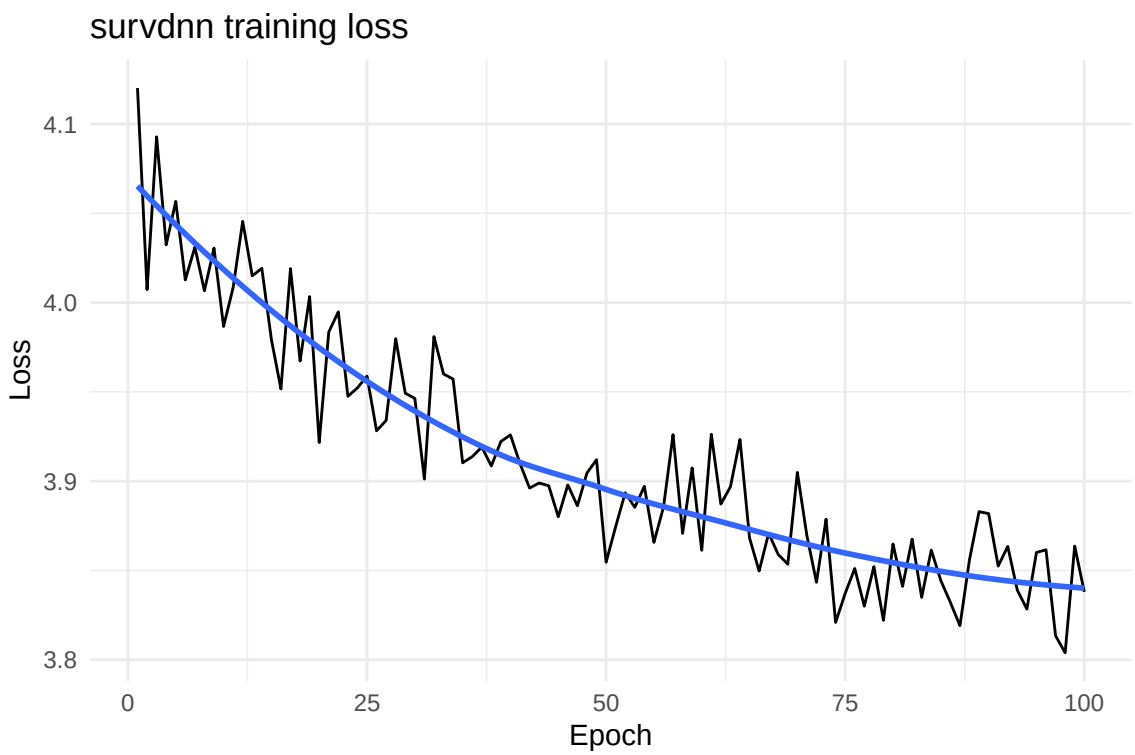
Example

A model is fit with the same formula interface as `survival::coxph()`, on the Veterans' Administration lung cancer trial data (`survival::veteran`).

```
library(survdnn)
library(survival)
library(ggplot2)

veteran <- survival::veteran

mod <- survdnn(
  Surv(time, status) ~ age + karno + celltype,
  data       = veteran,
  hidden     = c(16, 8),
  loss       = "cox",
  epochs     = 100,
  lr         = 1e-3,
  .seed      = 1,
  verbose    = FALSE
)

mod
```

`plot_loss()` reads the training-loss trajectory stored on the fitted object.

```
plot_loss(mod, smooth = TRUE)
```



The loss decreases steadily over the 100 epochs with no sign of divergence or premature plateauing, so this configuration (learning rate, epoch count, architecture) is a reasonable one for this dataset; a curve that is still falling steeply at epoch 100 would call for more epochs, and one that plateaus in the first 20 would call for fewer.

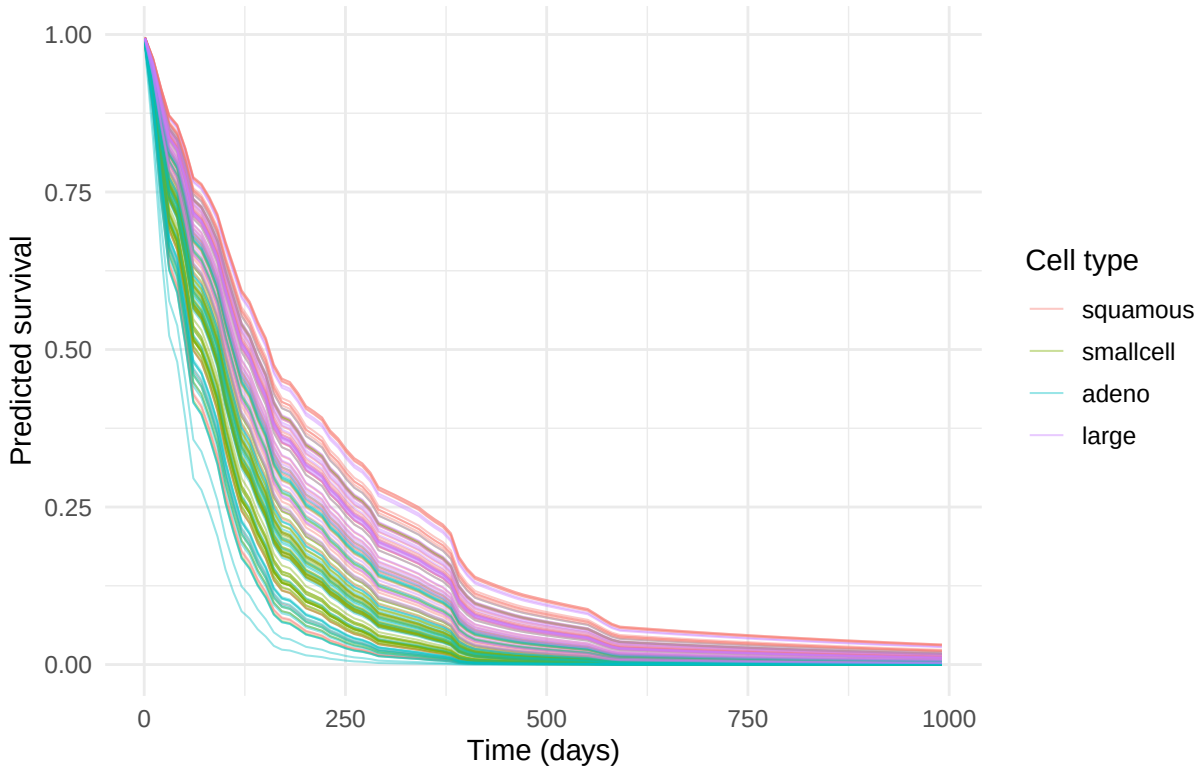
Survival curves at a grid of times follow the same `predict()` interface used for `coxph` and `survfit` objects.

```
times_eval <- c(30, 90, 180)
pred <- predict(mod, newdata = veteran, type = "survival", times = times_eval)
round(head(pred), 3)
```

```
   t=30  t=90 t=180
1 0.806 0.573 0.259
2 0.847 0.652 0.354
3 0.835 0.629 0.325
4 0.832 0.622 0.316
5 0.844 0.645 0.346
6 0.684 0.376 0.093
```

Each row is one patient's predicted survival probability at 30, 90, and 180 days; probabilities decrease left to right within every row, as they must for a valid survival curve. Plotting the full predicted curve for every patient, colored by cell type, shows the same information as a population, not just six numbers per patient.

```
times_grid <- seq(1, max(veteran$time), by = 10)
pred_grid <- predict(mod, newdata = veteran, type = "survival", times = times_grid)

curve_df <- data.frame(
  id = rep(seq_len(nrow(veteran)), each = length(times_grid)),
  time = rep(times_grid, nrow(veteran)),
  surv = as.vector(t(as.matrix(pred_grid))),
  celltype = rep(veteran$celltype, each = length(times_grid))
)

ggplot(curve_df, aes(time, surv, group = id, color = celltype)) +
  geom_line(alpha = 0.4, linewidth = 0.4) +
  theme_minimal(base_size = 12) +
  labs(x = "Time (days)", y = "Predicted survival", color = "Cell type")
```



Squamous and large-cell tumors sit above adeno and small-cell throughout follow-up, tracking

the known clinical ordering in this cohort (adenocarcinoma and small-cell histologies carry worse prognosis), evidence that the fitted risk score is picking up a real, clinically sensible signal rather than noise, ahead of the formal discrimination check below.

Model assessment

Discrimination and calibration are read off the same predicted matrix.

```
y <- model.response(model.frame(mod$formula, veteran))

c(
  cindex = cindex_survmat(y, pred, t_star = 180),
  ibs    = ibs_survmat(y, pred, times = times_eval)
)
```

```
cindex.c-index         ibs.ibs
      0.733631        0.177018
```

A C-index above 0.5 means the model ranks patients better than chance; the value here says that for a randomly chosen pair of patients, one still at risk at day 180 and one not, the model assigns the higher risk score to the right patient noticeably more often than a coin flip would. The IBS averages squared calibration error over the three horizons on the 0-1 probability scale, so smaller is better; a value in this range is unremarkable for a training-set evaluation on 137 patients and should be read alongside the cross-validated estimate below, which is not optimistic in the same way.

Cross-validated performance follows the same call structure, now with the model refit per fold.

```
res <- cv_survdnn(
  Surv(time, status) ~ age + karno + celltype,
  data    = veteran,
  times   = c(30, 90, 180),
  metrics = c("cindex", "ibs"),
  folds   = 3,
  .seed   = 42,
  hidden  = c(16, 8),
  epochs  = 50,
  verbose = FALSE
)

summarize_cv_survdnn(res)
```

```
# A tibble: 2 x 5
  metric  mean      sd lower upper
  <chr>  <dbl>   <dbl> <dbl> <dbl>
1 cindex 0.601 0.0515  0.542 0.659
2 ibs    0.209 0.00615 0.202 0.216
```

The cross-validated metrics are computed on held-out folds, so they are the honest estimate of out-of-sample performance; comparing them against the training-set numbers above is the standard check for optimism, a large gap between the two would indicate overfitting, which a 137-patient dataset with a 16-8 hidden architecture is genuinely at risk of, while a small gap supports the model generalizing rather than memorizing the training split.

Practical notes

The AFT and Cox-Time losses cost more per epoch than plain Cox: AFT adds one learnable global parameter, and Cox-Time evaluates the network on a full $n_e \times n$ grid of event-time and subject pairs at every step, so training cost grows with the number of events rather than staying constant. Batch normalization interacts poorly with very small batches; since `survdnn` trains full-batch by default this is only a concern for small datasets, where turning batch normalization off, `batch_norm = FALSE`, is often more stable than leaving it on. Dropout and weight decay are the two regularizers exposed directly, and `tune_survdnn()` and `gridsearch_survdnn()` search over these together with architecture and learning-rate settings under cross-validation.

The proportional-hazards structure of the Cox and Cox-L2 losses has a consequence worth flagging for any downstream curve-shape analysis: under either loss, $\hat{S}_i(t) = \hat{S}_0(t)^{\exp(\eta_i)}$ is a scalar power transform of one shared baseline curve, so any two predicted curves from the same fitted model differ only in scale, never in shape. Methods that cluster or compare predicted curves by their shape, such as `unsurv`, are only doing genuinely curve-native work when paired with `survdnn` fit under `"aft"` or `"coxtime"`, where curve shape is free to vary across individuals rather than being tied to a single common baseline.